

Especificação Técnica de Arquitetura: Yawara System Neural Architecture (Y-SNA)

Autor

Hélio Peres Martins Neto

Revisado Por

Data

01/12/2025

Resumo Executivo: Este documento apresenta a especificação técnica e arquitetural da **Yawara System Neural Architecture (Y-SNA)**, um subsistema de inteligência artificial desenvolvido para automatizar e otimizar a triagem de candidatos para o projeto de extensão Yawara MotoStudent da UFGD. A proposta detalha uma abordagem híbrida e evolutiva para Redes Neurais de Classificação, solucionando o problema de *Cold Start* (ausência inicial de dados históricos) através de uma fase determinística com injeção manual de pesos heurísticos, que transiciona organicamente para uma fase estocástica via *Transfer Learning*. O documento abrange a definição do stack tecnológico (Python, FastAPI, TensorFlow), o pipeline de ingestão de dados não estruturados (OCR/NLP), modelagem matemática para auto-regulação de dificuldade e protocolos rigorosos de governança de dados e transparência algorítmica (XAI).

PALAVRAS-CHAVE: Arquitetura de Software; Redes Neurais Artificiais; Processo Seletivo Automatizado; MotoStudent; UFGD; TensorFlow; Aprendizado por Transferência; IA Explicável (XAI); Engenharia de Dados.

Technical Architecture Specification: Yawara System Neural Architecture (Y-SNA)

Abstract: This document specifies the technical architecture and implementation protocols for the **Yawara System Neural Architecture (Y-SNA)**, an AI subsystem designed to automate and optimize candidate screening for the Yawara MotoStudent extension project at UFGD. The proposal details a hybrid evolutionary approach for Classification Neural Networks, addressing the *Cold Start* problem through a deterministic phase using manually injected heuristic weights, which organically transitions into a stochastic phase via *Transfer Learning*. The document covers the technology stack (Python, FastAPI, TensorFlow), the unstructured data ingestion pipeline (OCR/NLP), mathematical modeling for difficulty auto-regulation, and rigorous data governance protocols ensuring algorithmic transparency (XAI).

KEYWORD: Software Architecture, Artificial Neural Networks, Automated Recruitment, MotoStudent, UFGD, TensorFlow, Transfer Learning, Explainable AI (XAI), Data Engineering.

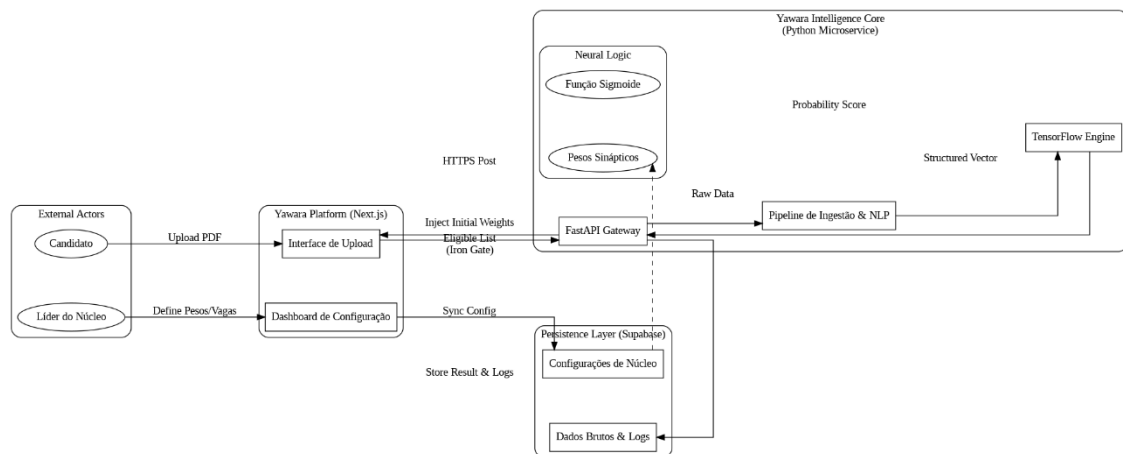
1. INTRODUÇÃO

Este documento estabelece as especificações técnicas, os protocolos de implementação e as diretrizes de governança do **Yawara System Neural Architecture (Y-SNA)**, um microsserviço de inteligência artificial projetado para atuar como o núcleo de processamento seletivo do projeto de extensão Yawara MotoStudent. Diferente de sistemas tradicionais de suporte à decisão, onde a máquina apenas sugere caminhos, o Y-SNA é arquitetado operacionalmente como um **Agente Externo de Validação**. Sua função crítica é estabelecer uma barreira técnica objetiva — conceitualmente denominada "Portão de Ferro" — que blindo o processo de recrutamento contra vieses cognitivos, favoritismos ou fadiga de avaliação humana. Ao restringir o acesso da banca examinadora exclusivamente aos candidatos que superam a *Baseline Probabilística de Competência* calculada pela rede, o sistema assegura matematicamente a isonomia e a impessoalidade do certame, fundamentando a elegibilidade em métricas de desempenho acadêmico normalizadas e auditáveis.

O principal desafio de engenharia endereçado por esta especificação reside no problema do "*Cold Start*" (partida a frio), caracterizado pela inexistência inicial de um *dataset* histórico rotulado que viabilize o treinamento supervisionado clássico de modelos profundos. Para solucionar esta limitação sem sacrificar a capacidade de evolução futura, o Y-SNA adota uma **Arquitetura Híbrida Evolutiva**. Na fase atual de operação (V1), o sistema opera em modo determinístico sobre uma infraestrutura estocástica: a topologia da Rede Neural é instanciada com pesos sinápticos "congelados", configurados manualmente para replicar a heurística da fórmula $EART \sum \eta \cdot c \cdot \omega$. Esta estratégia permite que o sistema seja implantado imediatamente com total explicabilidade, comportando-se como um algoritmo exato executado sobre uma matriz de tensores do TensorFlow, garantindo que cada decisão de corte possa ser decomposta e justificada logicamente perante a auditoria acadêmica.

A arquitetura prevê nativamente a transição para a inteligência preditiva (Fase V2) através de um pipeline de automação que independe de intervenção humana direta, mitigando riscos de manipulação. Conforme o projeto acumula dados de "*Ground Truth*" — métricas objetivas de desempenho técnico e comportamental dos membros admitidos ao final de cada ciclo —, o sistema executará rotinas autônomas de *Transfer Learning* e *Fine-Tuning*. Neste estágio maduro, a rede neural passará a detectar padrões não-lineares de sucesso que escapam à lógica heurística inicial, como a correlação entre disciplinas

oprativas específicas e a alta performance em núcleos distintos. Para viabilizar a sustentabilidade do projeto, todo o microserviço é containerizado e otimizado para execução em ambientes de nuvem de baixo custo ou gratuitos (como Render ou Oracle Cloud Free Tier), utilizando processamento assíncrono para maximizar a eficiência de recursos computacionais limitados, mantendo a conformidade estrita com a LGPD através de protocolos de anonimização irreversível nos dados de treinamento.

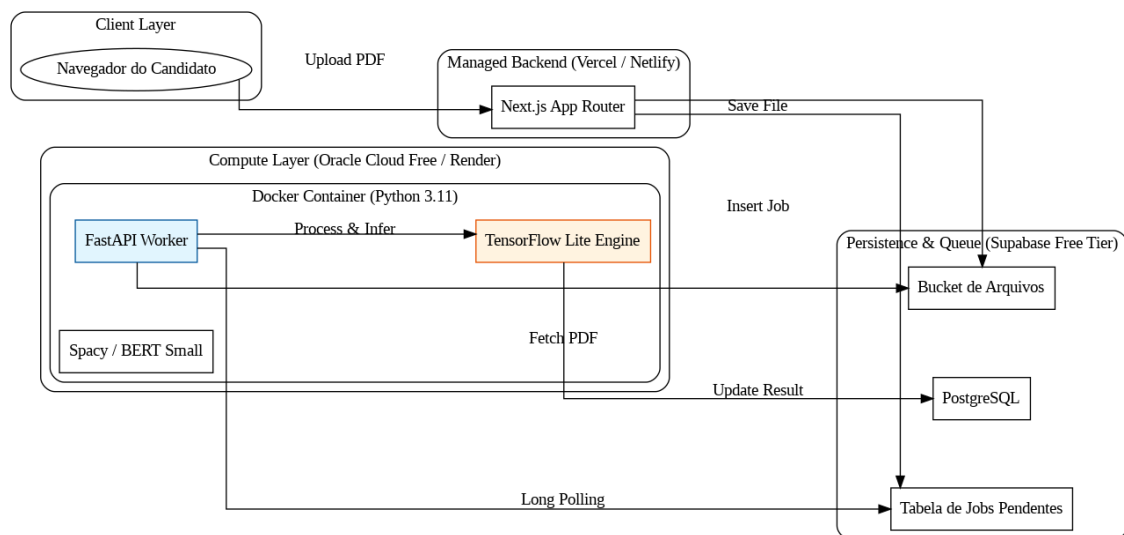


2. STACK TECNOLÓGICO

A materialização do Y-SNA exige um ecossistema tecnológico que equilibre o rigor matemático necessário para operações de *Deep Learning* com a sustentabilidade financeira de um projeto de extensão universitário. O núcleo de processamento é construído sobre a runtime **Python 3.11+**, selecionada não apenas por sua hegemonia na Ciência de Dados, mas pela maturidade de seu gerenciamento de memória em ambientes containerizados. Para expor as capacidades neurais como um serviço consumível, utiliza-se o framework **FastAPI**. A escolha desta tecnologia é estratégica: operando sobre o padrão ASGI (*Asynchronous Server Gateway Interface*), o FastAPI permite que o microserviço gerencie centenas de conexões de upload simultâneas de forma não-bloqueante, maximizando o *throughput* (vazão) mesmo em hardware com limitações severas de CPU, características típicas de planos de hospedagem gratuitos. O motor de inteligência artificial é alicerçado no **TensorFlow 2.x Lite**, uma versão otimizada da biblioteca padrão que permite a execução de inferência em modelos quantizados, reduzindo drasticamente a pegada de memória RAM necessária para carregar os tensores, viabilizando a operação em instâncias de nuvem com apenas 512MB ou 1GB de memória.

A estratégia de persistência de dados adota uma abordagem *Serverless* através do **Supabase**, que provê uma instância gerenciada de PostgreSQL. A arquitetura explora extensivamente o tipo de dado binário `JSONB` para armazenar os vetores de entrada não-estruturados e os logs de decisão da IA, eliminando a necessidade operacional e o custo de manter um banco NoSQL dedicado (como MongoDB). A segurança da camada de dados é delegada ao próprio banco através de *Row Level Security* (RLS), desonerando a aplicação Python de verificações de permissão redundantes e economizando ciclos de processamento. Para garantir a portabilidade e a imunidade a conflitos de dependências, todo o microsserviço — incluindo o interpretador Python, as bibliotecas de NLP e o modelo TensorFlow — é encapsulado em uma imagem **Docker** *multi-stage* otimizada (baseada em Alpine ou Slim Debian), resultando em um artefato final de tamanho reduzido, ideal para *deploys* rápidos e recuperação automática de falhas.

O modelo de implantação (*Deployment*) foi desenhado especificamente para a realidade de **Custo Zero ("Zero-Cost Architecture")**, utilizando a camada gratuita (*Free Tier*) de provedores de nuvem modernos como a **Oracle Cloud Infrastructure (OCI)** ou **Render**. A arquitetura privilegia o uso de instâncias baseadas em processadores **ARM (Ampere Altra)**, disponíveis gratuitamente na OCI com recursos superiores (até 4 OCPUs e 24GB de RAM) comparados às tradicionais instâncias x86. Para contornar a latência de *Cold Start* (tempo de inicialização) comum em serviços gratuitos que "adormecem" por inatividade, o sistema implementa um padrão de arquitetura orientada a eventos: o recebimento do PDF no front-end dispara um evento em uma fila de tarefas (gerenciada via Redis ou tabela de jobs no Supabase), que é consumida assincronamente pelo worker Python. Isso desacopla a experiência do usuário da disponibilidade imediata de recursos computacionais, garantindo que o processamento pesado da rede neural ocorra em segundo plano sem travar a interface, mantendo o sistema responsivo e financeiramente sustentável a longo prazo.



3. DEFINIÇÃO DE DADOS E INGESTÃO

A operacionalização do Y-SNA depende de um pipeline de engenharia de dados determinístico, capaz de converter documentos não estruturados em vetores matemáticos precisos, mitigando a alta entropia de leiautes variados entre instituições de ensino. O fluxo de ingestão é projetado para ser assíncrono e resiliente a falhas: o processo inicia-se com o recebimento do histórico escolar em formato PDF (*Input Raw*), que é imediatamente submetido a uma extração textual bruta. Para equilibrar a eficiência de custos com a precisão semântica, o sistema implementa uma arquitetura de **Inferência em Camadas**. A primeira linha de defesa é composta por um Motor de NLP Proprietário local, baseado em modelos leves (como BERT *distilled* ou spaCy otimizado), treinados especificamente para a tarefa de Extração de Entidades Nomeadas (NER) em documentos acadêmicos brasileiros. Este motor utiliza um **Dicionário Canônico de Disciplinas** — uma ontologia construída a partir da análise dos Projetos Pedagógicos de Curso (PPCs) — para normalizar variações textuais (ex: "Cálculo 1", "Matemática A") em chaves únicas padronizadas.

Nos casos em que o modelo local apresenta baixa certeza na identificação (confiança inferior a 85%), tipicamente devido a disciplinas optativas novas ou erros de OCR, o sistema aciona automaticamente um protocolo de *fallback* para **Modelos Fundacionais via API** (como Google Gemini Flash ou GPT-4o Mini). Nesta etapa, apenas o fragmento ambíguo do texto é enviado, utilizando a capacidade de raciocínio semântico da LLM

para resolver a normalização que o modelo local falhou em processar. Apenas em cenários extremos, onde ambas as camadas falham, a entidade é enviada para uma fila de curadoria humana. Esta estratégia assegura níveis de automação próximos a 100% com custo marginal, recorrendo a APIs externas apenas para exceções.

Após a resolução das entidades, os dados são consolidados e persistidos no banco de dados como um **Objeto de Dados Estruturado** em formato JSON. Este artefato atua como a "Fonte da Verdade" (*Source of Truth*) do sistema, contendo o histórico escolar limpo e metadados de auditoria. Para a alimentação da Rede Neural, este objeto é submetido a um processo de vetorização dinâmica: o algoritmo filtra apenas as disciplinas relevantes para o núcleo alvo (conforme configuração do líder), normaliza as notas para a escala linear [0, 1] e constrói o vetor numérico denso que servirá de *input* para o modelo matemático.

3.2. Objeto de Dados Estruturado (Pós-ETL) Este é o dado persistido no banco após a limpeza do PDF.

```
1. {  
  
  "transaction_id": "uuid-v4",  
  
  "candidate_id": "uuid-v4",  
  
  "cycle_id": "uuid-v4",  
  
  "processing_metadata": {  
  
    "engine_version": "v1.2-hybrid",  
  
    "fallback_triggered": true, // Indica se precisou de LLM externa  
  
    "ocr_confidence_mean": 0.98  
  
  },  
  
  "academic_record": [  
  
    {  
  
      "subject_canonical": "CALCULO_DIFERENCIAL_INTEGRAL_1",  
  
      "original_text_fragment": "MAT001 - Calc. Dif. Int. I",  
  
      "grade_raw": 8.5,           // Nota original (0-10 ou 0-100)  
  
      "grade_normalized": 0.85, // Normalizado para 0-1  
  
    }  
  
  ]  
}
```

```

    "workload_hours": 60,

    "source": "local_nlp_model",

    "confidence_score": 0.99

  },

  {

    "subject_canonical": "INTELIGENCIA_ARTIFICIAL",

    "original_text_fragment": "Tópicos Esp. em Comp. IV",

    "grade_raw": 9.0,

    "grade_normalized": 0.90,

    "workload_hours": 40,

    "source": "gemini_api_fallback", // Resolvido via API

    "confidence_score": 0.92

  }

]

}

```

3.3. Output de Inferência (Resposta da API Neural) Este é o retorno do microsserviço Python para o backend da aplicação.

```

1. {

  "model_version": "v1.0.0-deterministic",

  "nucleus_id": "uuid-v4",

  "prediction": {

    "probability_score": 0.924, // Saída da Sigmoide

    "is_eligible": true,      // score >= baseline_threshold

    "baseline_used": 0.70

  },

  "explainability": {

    // Vetor de contribuição para XAI (Explainable AI)

```



```
"top_contributors": [

  { "subject": "CALCULO_DIFERENCIAL_INTEGRAL_1", "impact": "+0.45" },

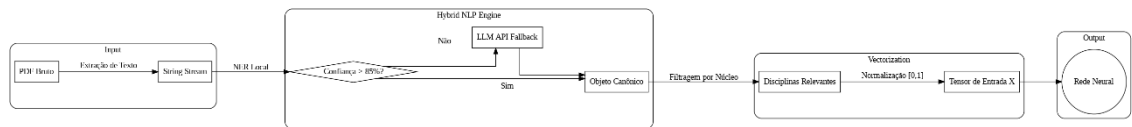
  { "subject": "FISICA_MECANICA", "impact": "+0.30" }

],

"missing_critical_subjects": []

}

}
```



4. ARQUITETURA E MODELAGEM MATEMÁTICA

A engenharia do Y-SNA fundamenta-se em uma topologia de rede neural dinâmica, projetada para realizar uma **Avaliação Multi-Núcleo Simultânea**. Diferente de classificadores binários simples que avaliam uma única hipótese, o sistema instância múltiplos contextos de inferência para "provar" o candidato contra os requisitos de todos os núcleos ativos do projeto. Matematicamente, isso é realizado através de uma estratégia de *Just-in-Time Instantiation*, onde a arquitetura se adapta à dimensionalidade variável de cada núcleo ($N_{features}$). Se o núcleo de Powertrain exige 5 disciplinas específicas e o de Gestão exige 3, o sistema constrói e executa grafos computacionais distintos para cada contexto, gerando ao final uma **Matriz de Elegibilidade Global** que mapeia as aptidões do candidato em todo o espectro do projeto.

Especificação da Topologia (Fase 1 - Determinística)

Para garantir a equivalência matemática exata com a heurística humana na fase de *Cold Start*, a rede opera sob uma arquitetura de **Perceptron de Camada Única (SLP)**. Neste estágio, a camada de entrada (o tensor de notas normalizadas) conecta-se diretamente a um neurônio de saída através de uma transformação linear seguida de uma não-linearidade. A inovação técnica reside na injeção determinística de conhecimento: a matriz de pesos sinápticos (ω) não é iniciada aleatoriamente, mas preenchida com os

valores exatos de importância definidos pelos líderes dos núcleos, enquanto o viés (**b**) é calculado estaticamente como o inverso do *Baseline Threshold* daquele núcleo específico. A função de ativação **Sigmoide** ($\sigma(z)$) é aplicada ao resultado, comprimindo a soma ponderada para o intervalo (0, 1). Esta configuração assegura que a rede se comporte, provisoriamente, como um algoritmo de soma ponderada auditável, mas rodando sobre a infraestrutura de tensores que permitirá a evolução futura.

Especificação da Topologia (Fase 2 - Estocástica/Profunda)

Para a fase de maturação, onde o sistema aprende com o desempenho real (*Ground Truth*), a arquitetura expande-se para um **Multilayer Perceptron (MLP)** capaz de capturar correlações não-lineares entre disciplinas. A topologia evolui para incluir uma camada oculta (*Hidden Layer*) dimensionada dinamicamente com a fórmula $H = \min(2N, 32)$ neurônios, onde **N** é o número de disciplinas de entrada, equipada com função de ativação **ReLU** (*Rectified Linear Unit*) para introduzir a não-linearidade necessária à detecção de padrões complexos.

Para garantir a generalização do modelo em cenários de dados escassos (*Small Data*), o treinamento adota protocolos rigorosos de regularização. Não será utilizado *Dropout* na camada oculta para preservar a integridade do sinal em vetores pequenos; em vez disso, aplica-se uma penalização L2 (*Weight Decay*) com fator $\lambda \approx 10^{-4}$ diretamente nos kernels. O processo de otimização é conduzido pelo algoritmo **Adam** com taxa de aprendizado inicial $\alpha \approx 10^{-3}$, monitorado por uma política de *Early Stopping* com paciência de 10 épocas baseada na perda de validação. A função de perda adotada é a **Binary Cross-Entropy**, ideal para penalizar divergências em classificações probabilísticas, assegurando que o sistema aprenda a distinguir sutilmente entre um candidato "apenas aprovado" e um "talento excepcional".

5. CICLO DE VIDA EVOLUTIVO DA REDE NEURAL

A arquitetura do Y-SNA foi projetada para transcender a estaticidade dos algoritmos de seleção tradicionais, incorporando mecanismos de adaptação dinâmica que asseguram a perenidade e a justiça do processo seletivo. Um risco inerente a sistemas de triagem baseados exclusivamente em histórico escolar é o "Viés de Senioridade", onde candidatos em semestres avançados são sistematicamente favorecidos em detrimento de calouros talentosos, meramente pelo maior volume absoluto de disciplinas cursadas. Para mitigar esta distorção e evitar a obsolescência dos critérios, o sistema implementa um protocolo de **Homeostase Algorítmica**. No intervalo entre ciclos seletivos, o núcleo de inteligência analisa estatisticamente a distribuição dos escores e as taxas de ocupação da edição anterior. Caso detecte que um núcleo específico apresentou uma taxa de aprovação inferior à meta de preenchimento de vagas — indicando uma barreira de entrada proibitiva —, o sistema aciona automaticamente uma rotina de Ajuste de Viés (*Bias Adjustment*).

Matematicamente, esta regulação macroscópica não altera os pesos sinápticos que definem a importância relativa das disciplinas, preservando a hierarquia de competências técnicas estabelecida pelos líderes. A intervenção ocorre exclusivamente no termo de viés (b) da equação neural, alterando sua magnitude para deslocar horizontalmente a curva de ativação da função sigmoide. Em termos práticos, se o sistema precisa "abrir a torneira" para preencher vagas, ele reduz a exigência da pontuação bruta acumulada, permitindo que candidatos com menor volume de créditos, porém com excelência acadêmica nas disciplinas fundamentais já cursadas, atinjam o limiar de elegibilidade. O mecanismo inverso é aplicado caso a seleção anterior tenha gerado um volume incontrolável de candidatos, garantindo que o sistema oscile sempre em torno de um ponto de equilíbrio ideal entre quantidade e qualidade de aprovados (relação alvo de 3 candidatos por vaga).

A evolução da inteligência preditiva ocorre através de um **Pipeline de Retreino Automatizado**, desenhado para operar sem intervenção humana direta. Diferente de fluxos de *Data Science* tradicionais que dependem de cientistas de dados para disparar treinamentos, o Y-SNA utiliza um agendador de tarefas (*Cron Job*) isolado no servidor de IA. O gatilho para este processo é a inserção dos dados de "Verdade Fundamental" (*Ground Truth*) ao final de cada ciclo do projeto, onde o desempenho real dos membros é quantificado. O *job* recupera os logs históricos anonimizados, reconstrói os tensores de entrada originais e executa o algoritmo de *Backpropagation*. Este processo ajusta os pesos

da rede para minimizar a discrepância entre a previsão inicial e a realidade observada, assegurando que a IA aprenda com o sucesso empírico dos membros na oficina, e não com as opiniões subjetivas da banca, eliminando progressivamente a perpetuação de vieses de afinidade.

6. GOVERNANÇA, AUDITORIA E TRANSPARÊNCIA

A integridade institucional do Y-SNA sustenta-se sobre um arcabouço de auditabilidade rigorosa, desenhado para converter a opacidade tradicional das redes neurais em uma arquitetura de "Caixa de Vidro". Em conformidade com os princípios de administração pública e acadêmica, o sistema implementa protocolos de **Não-Repúdio** e **Imutabilidade** para todos os atos decisórios. Cada inferência realizada pela inteligência artificial gera um artefato digital permanente em um *ledger* (livro-razão) de logs analíticos. Este registro captura não apenas o veredito final de elegibilidade, mas o *snapshot* forense exato do estado do sistema no milissegundo da decisão, incluindo os dados brutos extraídos, o vetor de pesos configurado pelo líder e, crucialmente, o *hash* da versão do modelo utilizado.

Para assegurar a rastreabilidade e a capacidade de reversão (*Rollback*) em caso de falhas, o ciclo de vida dos modelos neurais segue estritamente o padrão de **Versionamento Semântico** (SemVer). A Fase Determinística opera sob versões da série $v1.x.x$, onde incrementos representam ajustes finos nos pesos heurísticos, devidamente catalogados com a justificativa do líder responsável. A transição para a Fase Estocástica inaugura a série $v2.0.0$, onde cada nova iteração treinada automaticamente recebe uma tag única. Isso permite que a auditoria reconstrua com precisão matemática a lógica que governava o "Portão de Ferro" em qualquer data passada, eliminando dúvidas sobre alterações de critérios *post-mortem* ou favorecimentos não documentados.

O compromisso com a transparência pedagógica é atendido através da incorporação nativa de princípios de **Explainable AI (XAI)**. O sistema não se limita a emitir um escore de probabilidade abstrato; ele processa os gradientes de ativação da rede para decompor a influência de cada variável no resultado final. Isso permite que a devolutiva ao candidato reprovado seja fundamentada em dados concretos, transformando a rejeição em um roteiro de estudos personalizado que aponta quais competências específicas precisam ser desenvolvidas. No âmbito da segurança e

privacidade, a arquitetura adota a **Lei Geral de Proteção de Dados (LGPD)** como requisito não-funcional crítico. Enquanto a segurança transacional é garantida por políticas de *Row Level Security* (RLS) no banco de dados, o pipeline de treinamento implementa um protocolo de **Anonimização Irreversível**. Ao final de cada ciclo seletivo, os vínculos entre os dados acadêmicos e a identidade civil dos candidatos são criptograficamente rompidos, preservando a riqueza estatística necessária para a evolução da IA, mas garantindo o absoluto sigilo e o direito ao esquecimento dos estudantes.